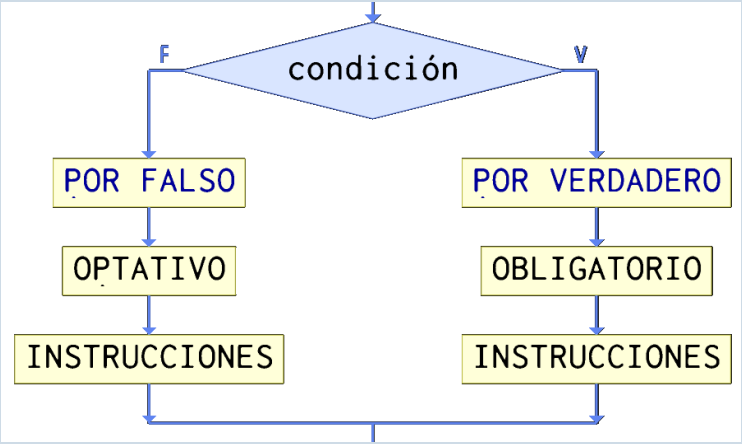
Programación I

Estructura de decisión

Decisión simple

La decisión simple nos permite que nuestro programa pueda evaluar una o más condiciones y elegir un posible curso de acción.



Diagrama

```c
if (condición){
  /* Instrucciones si la
  condición es verdadera*/
}
else{
  /* Instrucciones si la
  condición es falsa*/
}
```

Código C

Proposición lógica

Una condición se establece mediante una proposición lógica. La misma puede tener uno de dos posibles resultados: **verdadero** o **falso**.

Proposición lógica	Resultado
N > 0	verdadero
N != 10	falso
N == 10	verdadero
N < 10	falso

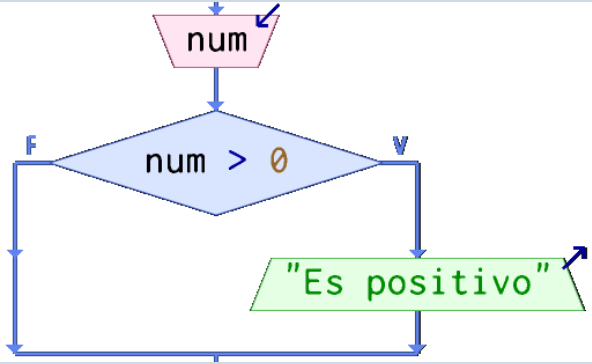
Siendo N igual a 10

Las palabras reservadas para representar verdadero y falso en C son:

verdadero → **true**
falso → **false**

Decisión simple - Ejemplo

Determinar si un número es positivo es simplemente preguntar si dicho número es mayor a cero.



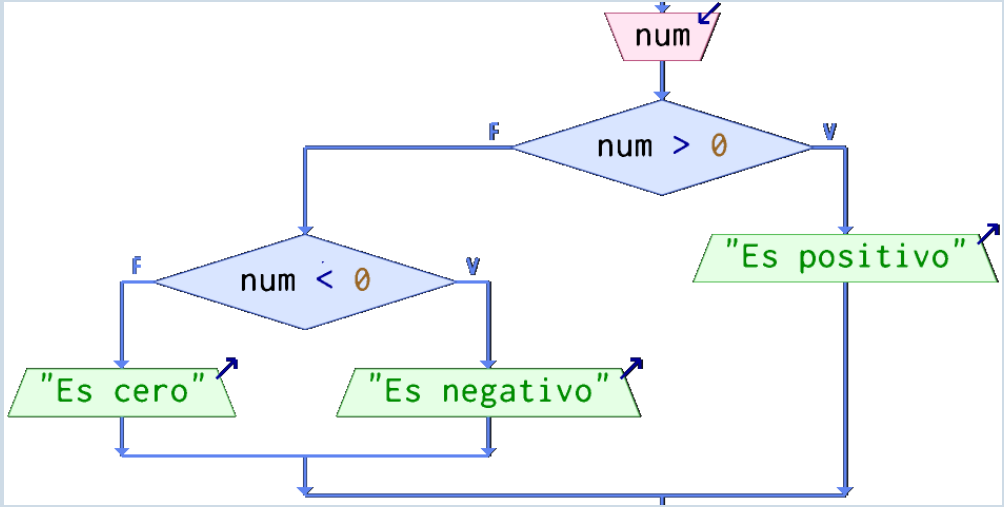
Diagrama

```
cin >> nro;
if (nro > 0){
  cout << "Es positivo";
}
```

Código C++

Decisión anidada - Ejemplo

Determinar si un número es positivo, negativo o cero requiere de más de una estructura de decisión. Éstas puedan anidarse sin problemas.



Diagrama

```
cin >> nro;
if (nro > 0){
  cout << "Es positivo";
}
else{
  if (nro < 0){
    cout << "Es negativo";
  }
  else{
    cout << " Es cero";
  }
}
```

Código C++

Operadores lógicos

Nos permiten realizar más de una proposición lógica en una misma estructura de decisión. Los más comunes son: **and** y **or**.

Operador AND
Exige que todas las proposiciones sean verdaderas para que el resultado de la condición sea verdadero.

&&

Operador OR
Exige que alguna de las proposiciones sea verdadera para que el resultado de la condición sea verdadero.

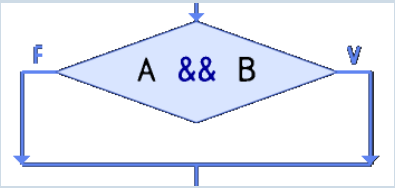
||

Operador NOT
Invierte el valor lógico de una condición, transformando una proposición verdadera en falsa y una falsa en verdadera.

!

Operador lógico **AND**

Exige que todas las proposiciones sean verdaderas para que el resultado de la condición sea verdadero.



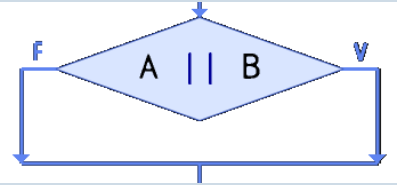
Diagrama

A	B	A && B
verdadero	verdadero	verdadero
verdadero	falso	falso
falso	verdadero	falso
falso	falso	falso

Tabla de verdad

Operador lógico **OR**

Exige que alguna de las proposiciones sea verdadera para que el resultado de la condición sea verdadero.



Diagrama

A	B	A || B
verdadero	verdadero	verdadero
verdadero	falso	verdadero
falso	verdadero	verdadero
falso	falso	falso

Tabla de verdad

Operador lógico **AND** - Ejemplo

Para considerar sospecha de Laravirus es necesario tener temperatura > 37.5 y dolor de cabeza.

```
float temp;
char  dc;
cin >> temp;
cin >> dc;
if (temp > 37.5 && dc == 'S'){
  cout << "Posible Laravirus";
}
else{
 cout << "Todo ok";
}
```

Código C++

temp > 37.5	dc == 'S'	temp > 37.5 && dc == 'S'
verdadero	verdadero	verdadero
verdadero	falso	falso
falso	verdadero	falso
falso	falso	falso

Tabla de verdad

Operador lógico **OR** - Ejemplo

Para considerarse de riesgo para Laravirus es necesario tener edad mayor a 80 o estar inmunosuprimido.

```cpp
int edad;
char inm;
cin >> edad;
cin >> inm;

if (edad > 80 || inm == 'S'){
  cout << "Riesgo de Laravirus";
}
else{
 cout << "Todo ok";
}
```

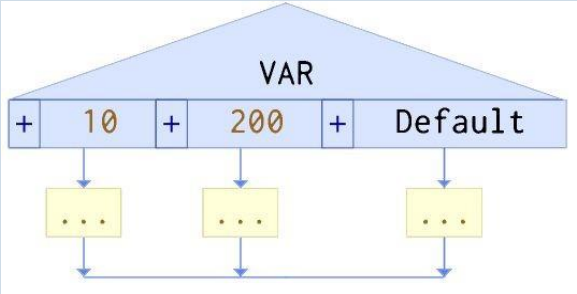
Código C++

edad > 80	inm == 'S'	edad> 80 || inm== 'S'
verdadero	verdadero	verdadero
verdadero	falso	verdadero
falso	verdadero	verdadero
falso	falso	falso

Tabla de verdad

Decisión múltiple

La decisión múltiple nos permite que nuestro programa pueda evaluar una variable y ejecutar un curso de acción a partir de su valor.



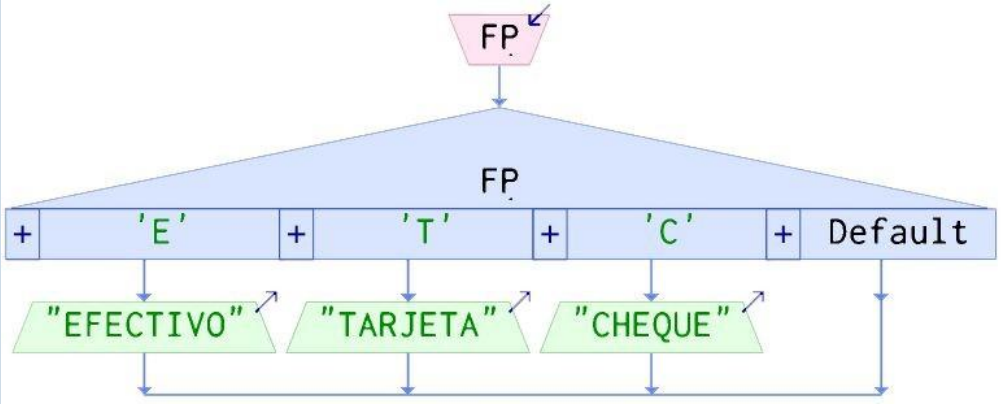
Diagrama

```
switch(var){
  case 10:
    /* Instrucciones si
    var es igual a 10 */
  break;
  case 100:
    /* Instrucciones si
    var es igual a 100 */
  break;
  default:
    /* Instrucciones si
    ningún caso se cumple */
  break;
}
```

Código C++

Decisión múltiple - Ejemplo

Mostrar la forma de pago de un cliente a partir de la inicial en mayúsculas de la misma. Las formas de pago pueden ser: Efectivo, Tarjeta o Cheque.



Diagrama

```
char fp;
cin >> fp;

switch(fp){
  case 'E':
    cout << "Elegiste efectivo";
  break;
  case 'T':
    cout << "Elegiste tarjeta";
  break;
  case 'C':
    cout << "Elegiste cheque";
  break;
}
```

Código C++

Créditos

- Angel Ruben Simón, docente UTN FRGP
- Brian Lara Campos, docente UTN FRGP